ARCADIAN-INTEGRATION-HANDOVER

Arcadian Outfitters Integration Handover Document

1. Executive Summary

Arcadian Outfitters runs NetSuite ERP as its system of record for customers, products, pricing, inventory, and orders. Makro Agency built a middleware connector called **Makro.Integration** that automatically synchronizes data between NetSuite and the Shopify B2B storefront.

The integration keeps the following data in sync: companies, parent-level customer contacts, company locations, location-level customer contacts, warehouse (inventory) locations, products, inventory levels, price lists, and catalog-to-location assignments. Orders placed on Shopify flow back into NetSuite as Sales Orders in real time.

Data flows through two distinct sync patterns:

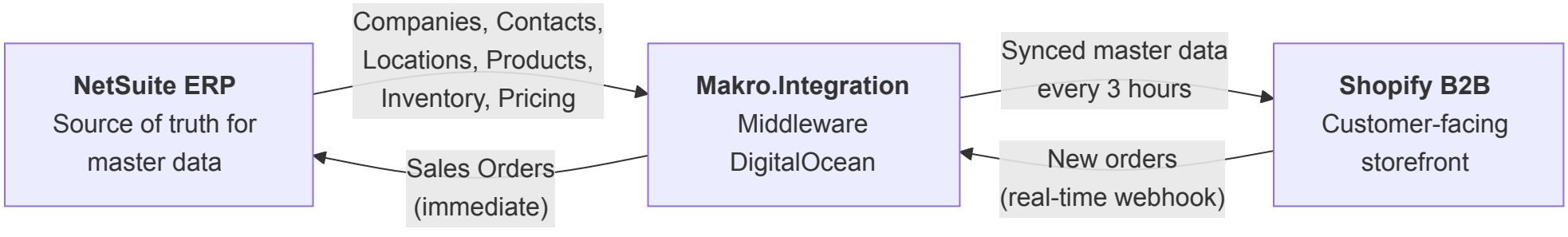
Pattern	What It Does	Frequency	Direction
Scheduled Pipeline	9 jobs run sequentially: Companies, Parent Customers, Company Locations, Company Customers, Inventory Locations, Products, Inventory Levels, Price Lists, Catalogs	Every 3 hours	NetSuite to Shopify
Real-Time Order Webhook	When a B2B customer places an order on Shopify, it is immediately sent to NetSuite as a Sales Order	Instant (real-time)	Shopify to NetSuite

The integration portal is hosted on a DigitalOcean droplet at `159.223.142.164` and provides monitoring, manual triggers, and error investigation for all sync operations. **The portal's public URL is pending — Arcadian Outfitters will provide a subdomain** (e.g., `integration.arcadianoutfitters.com`) that will resolve to the portal once DNS is configured. Until then, the portal is reachable over the server IP only; once the subdomain is live, it will be used everywhere this document currently shows a URL placeholder.

---

2. Integration Architecture

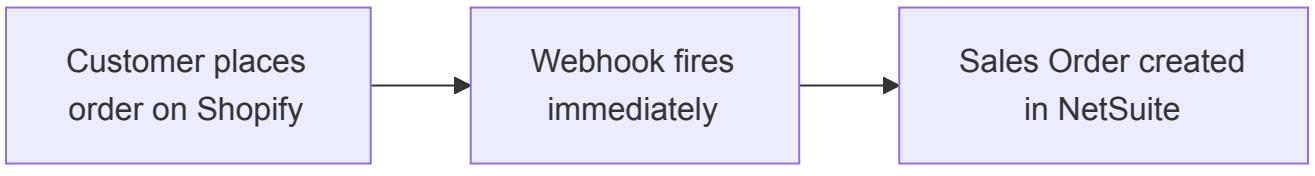
2.1 System Overview



Orders are one-way only: Shopify to NetSuite. Arcadian's connector does not pull orders from NetSuite, does not mirror NetSuite order status back to Shopify, and does not write picked / packed / shipped / invoiced state onto Shopify order metafields. Once a Sales Order is created in NetSuite, its lifecycle is managed inside NetSuite and is not reflected back on the Shopify side. Order fulfillment and status visibility for operations and customers happens in NetSuite, not Shopify.

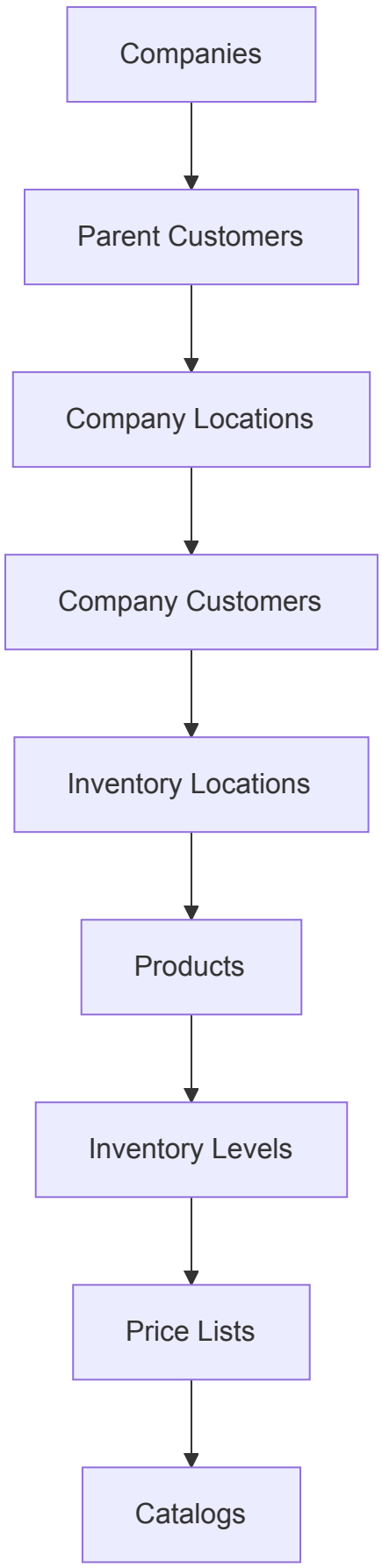
2.2 Data Flow by Sync Pattern

Order Webhook — Real-Time



When a Shopify order is placed, Shopify POSTs the order payload to the middleware endpoint `/api/shopify/order-created`. The middleware queues a background job (3 retries, 10-minute timeout per attempt) that translates the Shopify order into a NetSuite Sales Order payload and calls the NetSuite RESTlet to create it. The customer record on the Sales Order is resolved by following this ladder: (1) `ns_id` note attribute if present; (2) `loc_id` note attribute mapped back to the NetSuite child customer; (3) `order.company.location_id` mapped via the integration; (4) Shopify customer ID mapped via the integration.

Scheduled Pipeline — Every 3 Hours



2.3 Entity Sync Summary

Entity	Direction	Frequency	Sync Pattern
Companies	NetSuite to Shopify	Every 3 hours	Pipeline (Step 1)
Parent Customers (company-level contacts)	NetSuite to Shopify	Every 3 hours	Pipeline (Step 2)
Company Locations	NetSuite to Shopify	Every 3 hours	Pipeline (Step 3)
Company Customers (location-level contacts)	NetSuite to Shopify	Every 3 hours	Pipeline (Step 4)
Inventory Locations (Warehouses)	NetSuite to Shopify	Every 3 hours	Pipeline (Step 5)
Products	NetSuite to Shopify	Every 3 hours	Pipeline (Step 6)
Inventory Levels	NetSuite to Shopify	Every 3 hours	Pipeline (Step 7)
Price Lists	NetSuite to Shopify	Every 3 hours	Pipeline (Step 8)
Catalogs (location-to-pricelist assignment)	NetSuite to Shopify	Every 3 hours	Pipeline (Step 9)
Orders	Shopify to NetSuite	Real-time	Webhook

---

3. Entity Sync Reference

Cross-cutting behaviors

A handful of behaviors apply to every sync job below and are not repeated per entity:

- **Concurrency guard.** Each sync job checks `job_queues` for an existing in-progress run with the same name before it starts. If one is found, the new invocation **silently exits without logging a failure**. This protects against overlapping runs when the pipeline is triggered manually at the same moment the scheduler fires. Ops may see a manual-trigger banner ("Triggered successfully") that does not produce new data — this is the expected behaviour when a prior run is still executing.
- **Pagination.** Each sync job that reads list-style data from NetSuite paginates in chunks (typically 1,000 records per NetSuite query, with a short delay between pages to avoid rate-limit issues). Progress metrics on the Jobs page update as each chunk completes, not only at end-of-run.
- **Logging granularity.** Every record processed is written to the `job_record_logs` table with its outcome (`written`, `updated`, `skipped`, or `failed`). Failed records are additionally mirrored into `job_failure_logs` and surfaced on the portal's Errors page.
- **Failure notification.** A `JobFailureNotifier` email is sent only when a job finishes with a top-level failure (not for individual-record failures within a partially successful run).

3.1 Companies

What it does: Pulls parent-level customers from NetSuite and creates or updates corresponding Companies in Shopify. Each qualifying NetSuite parent customer becomes a Shopify Company. The company's active/inactive status is stored as a metafield so it is visible to the operations team inside the Shopify admin. Immediately after each company is created or updated, the integration also attempts to attach a primary contact at company level using the parent customer's notification email and primary-contact name.

Direction and Frequency: NetSuite to Shopify, every 3 hours (Pipeline Step 1)

Dependency: None. This is the first job in the pipeline.

Filter Conditions

Condition	Meaning
Customer is active	`isinactive = 'F'` — inactive customers are excluded
Parent-level only	Customer has no parent, or the companyname contains "Parent Company" (captures explicit parent-company records)
Not a placeholder	Companyname does not start with `zz.` (and, under the Parent Company branch, the parent companyname also does not start with `zz.`)
Not the excluded record	Customer ID is not 4241 (internal exclusion)
Entity status	Status is not "Qualified" and not "Unqualified", and the status itself is not inactive — leads and prospects are excluded
Sales rep filter	Salesrep's first name is not "VIEGEAR" (or the customer has no salesrep) — VIEGEAR-owned records are excluded

Data Mapping

NetSuite Field	Shopify Field	Notes
`companyname`	Company Name	
`id`	External ID	Used to link records across systems
`isinactive`	Metafield `netsuite:status`	Carries the NetSuite active/inactive flag onto the Shopify record
`custentity_2663_email_address_notif`	Primary contact email	Used when attempting to attach the primary contact after the company is saved
`custentity_prim_cont`	Primary contact first/last name	The value is split on whitespace; the first token becomes firstName and the remainder becomes lastName

---

3.2 Parent Customers

What it does: For each synced Company, retrieves the related NetSuite contacts and attaches them to the Shopify Company as company-level customers (buyers). This step populates company-scoped buyers who can transact on behalf of the whole company rather than a single location. A follow-up step inside this job also assigns the contact the "Ordering only" permission across all of the company's locations, where that role exists.

Direction and Frequency: NetSuite to Shopify, every 3 hours (Pipeline Step 2)

Dependency: Companies must sync first. Parent Customers are attached to an already-synced Shopify Company.

Filter Conditions

Condition	Meaning
Scoped to a synced company	For each Shopify Company in the mapping table, the NetSuite query is run with that company's NetSuite ID as the scope
All contacts returned	No additional filter is applied on the contact query — every NetSuite contact linked to the parent customer record is considered

Before a contact is created, the integration walks a **three-step resolution ladder** to avoid duplicates:

1. **Mapping table — by source ID.** If a Parent Customer mapping already exists for this NetSuite customer, reuse it.
2. **Mapping table — by email (case-insensitive).** Check whether any existing Parent Customer or Company Customer mapping stores this email in its metadata; if so, reuse that Shopify customer.
3. **Live Shopify lookup.** As a last resort, query Shopify's Company Contacts API directly for a contact on this company matching the email. If found, the mapping table is backfilled so subsequent runs skip the live lookup.

If none of the three steps match, a new Shopify Company Customer is created and the mapping is written.

Data Mapping

NetSuite Field	Shopify Field	Notes
`contact.entityid` ("fullname")	First Name + Last Name	Split on whitespace; first token to firstName, remainder to lastName
`contact.email` (falls back to `customer.email`)	Email	Contact-level email preferred; customer-level email used when the contact's email is blank
`contact.phone`	Phone	Only sent if present; omitted otherwise
`customer.id`	Metafield `netsuite:company_id`	Links the Shopify contact back to the NetSuite company
`contact.isinactive`	Metafield `netsuite:status`	Active/inactive flag

3.3 Company Locations

What it does: For each synced Company, retrieves child customers (locations) from NetSuite and creates them as Company Locations in Shopify. Each child customer record represents a physical or billing location operating under the parent company. After syncing, if the company has at least one NetSuite child location, Shopify's auto-created default location (named the same as the company) is removed to avoid duplicates; if the company has no child locations, the default location is instead updated with the parent company's address, payment terms and contact.

Direction and Frequency: NetSuite to Shopify, every 3 hours (Pipeline Step 3)

Dependency: Companies must sync first. Company Locations are attached to their parent Shopify Company.

Filter Conditions

Condition	Meaning
Is a child customer	`parent IS NOT NULL` and the parent matches the synced company's NetSuite ID
Active	`isinactive = 'F'`
Has a sales rep	`salesrep IS NOT NULL` — a child customer with no assigned salesrep is excluded from the sync entirely
Entity status	Status is not "Qualified" and not "Unqualified", and the status itself is not inactive
Sales rep filter	Salesrep's first name is not "VIEGEAR"

Payment Terms Mapping

NetSuite's "days until net due" on each location's term is mapped to the closest Shopify payment-terms template:

NetSuite Days Until Due	Shopify Payment Terms
7	Net 7
15	Net 15
30	Net 30
45	Net 45
60	Net 60
90	Net 90
Null or 0 or less	**Net 30** (default fallback)

If the NetSuite value falls between two templates it is rounded to the nearest match.

Data Mapping

NetSuite Field	Shopify Field	Notes
`companyname`	Location Name	
`id`	External ID	
Shipping address (`addr1` , `addr2` , `city` , `zip` , `state` , `country` , `addrphone`)	Shipping Address	Populated from the NetSuite default shipping address. Carriage returns and newlines in each field are stripped and collapsed to spaces before sending.
Billing address	Billing Address	Populated from the NetSuite default billing address.
`term.daysuntilnetdue`	Payment Terms	Mapped to the nearest Shopify template (see table above)
`isinactive` (location_status)	Metafield `netsuite:status`	Active/inactive flag
`custentity_on_hold`	Metafield `netsuite:on_hold`	On-hold flag; defaults to `F` if NetSuite returns no value
`parent` (company_id)	Metafield `netsuite:parent_id`	Links the location back to its parent Company

Main Contact Assignment

After each Company Location is synced, the middleware assigns the parent company's **main contact** to every location under that company with the "Ordering only" role.

Default Shopify Location Cleanup

Shopify automatically creates one default Company Location per Company. The integration cleans this up as follows:

- If NetSuite returns **one or more** child locations for the company, the auto-created default (identified by its name matching the company name, or the single non-synced location remaining) is **deleted** to avoid duplicates.
- If NetSuite returns **zero** child locations, the default is **updated in place** with the parent company's address, payment terms, metafields, and contact — and a `CompanyLocation` mapping is written using the parent company's NetSuite ID as the `source_id` .

3.4 Company Customers

What it does: For each synced Company Location, retrieves the NetSuite contacts attached to that child customer and creates them as Shopify Company Customers at the location level. These are the day-to-day buyers who log in, order against a specific location, and see pricing and payment terms for that location.

Direction and Frequency: NetSuite to Shopify, every 3 hours (Pipeline Step 4)

Dependency: Company Locations must sync first. Each contact is attached to an already-synced Shopify Location.

Filter Conditions

Condition	Meaning
Scoped to a synced location	For each Shopify Location in the mapping table, the NetSuite query is run with that location's NetSuite ID as the scope

Condition	Meaning
All contacts returned	No additional filter is applied on the contact query — every NetSuite contact linked to the child customer is considered

As with Parent Customers, the integration walks a three-step resolution ladder before creating a contact: (1) mapping table by NetSuite source ID, (2) mapping table by email (case-insensitive, checking both Parent Customer and Company Customer rows), (3) live Shopify Company Contacts lookup. Only if all three miss is a new contact created.

Data Mapping

NetSuite Field	Shopify Field	Notes
`contact.entityid` ("fullname")	First Name + Last Name	Split on whitespace; first token to firstName, remainder to lastName
`contact.email` (falls back to `customer.email`)	Email	Contact-level email preferred; customer-level email used when the contact's email is blank
`contact.phone`	Phone	Only sent if present; omitted otherwise
`customer.id`	Metafield `netsuite:company_id`	Links the Shopify contact back to the NetSuite company
`contact.isinactive`	Metafield `netsuite:status`	Active/inactive flag

3.5 Inventory Locations (Warehouses)

What it does: Syncs all active NetSuite locations into Shopify as inventory locations. These represent the physical warehouses where Arcadian holds stock.

Direction and Frequency: NetSuite to Shopify, every 3 hours (Pipeline Step 5)

Dependency: None. This job runs independently of the customer-side jobs above, though it is sequenced after them in the pipeline so that Products (Step 6) has its warehouse locations ready.

Filter Conditions

Condition	Meaning
Active	`isinactive = 'F'` — inactive locations are excluded

No other filters are applied.

Data Mapping

NetSuite Field	Shopify Field	Notes
`fullname`	Location Name	
`id`	Metafield `netsuite:external_id`	

3.6 Products

What it does: Syncs active inventory items from NetSuite as Shopify Products. **Arcadian does not use product variants** — each NetSuite item becomes one simple Shopify product with no size/colour/option variations. A simple Shopify product still has a single "default variant" internally (this is how Shopify stores the SKU, barcode, and price for a simple product), but from a merchandising standpoint you will not see multiple options under any product in the storefront. Products are activated at every synced warehouse location so inventory can be tracked per warehouse on the next step. At the end of the job, any product that was previously synced but did not appear in the latest read from NetSuite is moved to Draft status in Shopify, so the storefront never displays stale items.

Direction and Frequency: NetSuite to Shopify, every 3 hours (Pipeline Step 6)

Dependency: Inventory Locations must sync first. Each newly created product is activated at all synced warehouse locations.

Filter Conditions

Condition	Meaning
Active	`item.isinactive = 'F'` — inactive items are excluded
Product class	`class.fullname != 'Vie Gear'` — items in the Vie Gear class are excluded

Data Mapping — Product

NetSuite Field	Shopify Field	Notes
`displayname` (as `title`)	Product Title	Set on create only
`description`	Description HTML	Set on create only
`isinactive`	Status	`F` → ACTIVE; anything else → DRAFT
`class.fullname`	Product Type	If the class name contains a colon, the portion **after** the colon is used as Shopify's Product Type; if there is no colon, the class "name" field is used instead
`id`	Metafield `netsuite:external_id`	Used to link records across systems
`unitstype`	Metafield `netsuite:weight`	NetSuite "unit type" value
`weightunits`	Metafield `netsuite:weight_unite`	NetSuite "weight units" value
`class` name portion	Metafield `netsuite:type_name`	Portion of `class.fullname` before the colon (or the full class path if no colon)
`class` state portion	Metafield `netsuite:type_state`	Portion of `class.fullname` after the colon (or the class "name" field if no colon)
`custitem_discontinued`	Shopify tag `discontinued`	If NetSuite marks the item as `T` (discontinued), the Shopify product gets a `discontinued` tag; if `F`, the tag is removed on every sync
(hardcoded)	`inventoryItem.tracked` = true	Every product is set to track inventory in Shopify; this cannot be turned off per product

Data Mapping — SKU / Price / Barcode (stored on the product's default variant)

NetSuite Field	Shopify Field	Notes
`itemid`	SKU	
`upccode`	Barcode	

3.7 Inventory Levels

What it does: Updates inventory quantities in Shopify to match NetSuite stock levels per warehouse. The job walks the list of products already synced by Step 6, queries NetSuite for their on-hand quantity at each warehouse, and writes those quantities into Shopify as absolute "on hand" levels (not incremental adjustments). Negative NetSuite quantities are normalized to zero before being written.

Direction and Frequency: NetSuite to Shopify, every 3 hours (Pipeline Step 7)

Dependency: Products and Inventory Locations must both sync first. Inventory levels reference product and location mappings created by those earlier jobs.

Filter Conditions

Condition	Meaning
Product must be mapped	Only products that already have an integration mapping are processed; newly added NetSuite items are skipped until the Products job picks them up
Location must be mapped	Only warehouses that already have an integration mapping are processed
Product variant record must exist	The Products sync stores each product's internal variant identifier in the mapping table; if that identifier cannot be read, the row is skipped (this is a technical prerequisite — Arcadian itself does not use product option variants)
Negative quantities	Clamped to 0 before writing to Shopify

Arcadian-specific "CA pool" rule

Arcadian separates Canadian-specific inventory from the rest of the catalog. SKUs that contain `CA-` or a literal `.` (period) are treated as CA-only products. The middleware enforces this rule by publishing CA-only SKUs' inventory only at the CA-designated warehouse and forcing the quantity to 0 at every other warehouse.

Data Mapping

NetSuite Field	Shopify Field	Notes
`quantityonhand` (per location)	Inventory Level (per location)	Set as an absolute "on_hand" quantity

---

3.8 Price Lists

What it does: Syncs NetSuite price levels into Shopify Price Lists. Each active price level in NetSuite becomes a Shopify Price List. This job sets up the Price List itself — name, currency, and default adjustment configuration. The per-product prices inside each Price List are loaded by the next job, Catalogs.

Direction and Frequency: NetSuite to Shopify, every 3 hours (Pipeline Step 8)

Dependency: Products should sync first so that when Catalogs (Step 9) writes fixed prices, those products already exist in Shopify.

Filter Conditions

Condition	Meaning
Active	`isinactive = 'F'`
Excluded	Name is not "Markup" — the internal Markup price level is skipped

Data Mapping

NetSuite Field	Shopify Field	Notes
`pricelevel.name`	Price List Name	

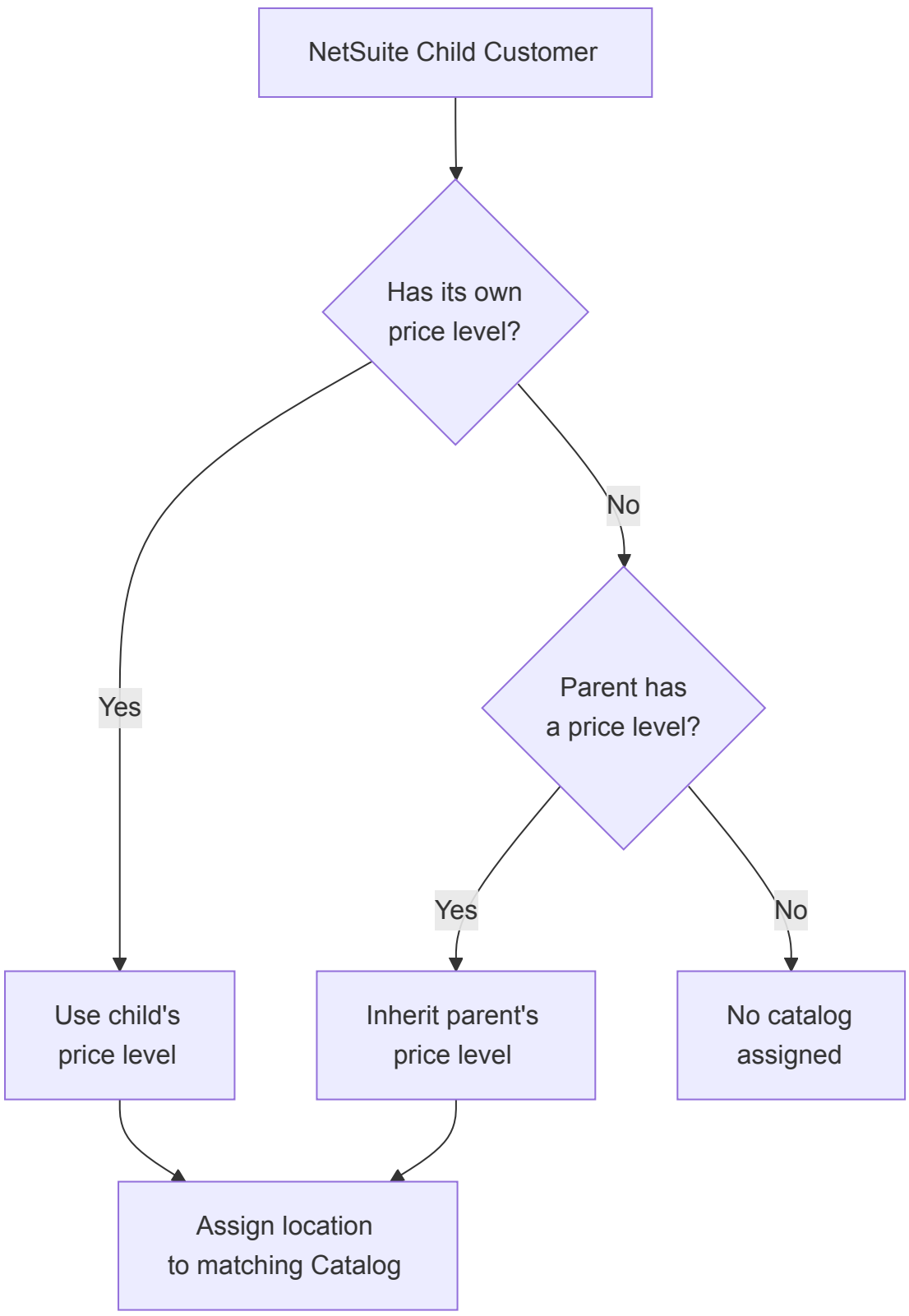
3.9 Catalogs

What it does: This is the job that connects Price Lists to Company Locations. For each Price List, it creates (or updates) a Shopify Catalog, assigns the Catalog to every Company Location that should see it, and loads the per-product fixed prices from NetSuite into the underlying Price List. This is what determines both *which* prices a given location sees on the B2B storefront and *what those prices are*.

Direction and Frequency: NetSuite to Shopify, every 3 hours (Pipeline Step 9)

Dependency: Price Lists AND Company Locations must both sync first. Catalogs reference both.

How Price Level Assignment Works



- If a child customer (location) has its own price level in NetSuite, that price level drives which Catalog the location is assigned to.
- If the child has no price level, it inherits the parent company's price level.
- Each NetSuite price level maps to exactly one Shopify Catalog, backed by the matching Price List.

Filter Conditions

Condition	Meaning
Active child customer	`isinactive = 'F'` — inactive customers' price-level assignments are ignored
Has an effective price level	A child or parent price level must be set; assignments with no effective price level are not mapped to a Catalog
Location resolves to Shopify	The NetSuite location must have an existing Company Location mapping in Shopify; unmapped locations are skipped

Per-Product Pricing Within Catalogs

After the Catalog is created or updated, the job reads all product prices from NetSuite for that price level and writes them as fixed prices into the underlying Shopify Price List. This ensures the Catalog reflects the correct per-product pricing for each assigned location.

New Catalog Publication

When a brand-new Catalog is created (first time a price level is seen), the middleware also creates a matching **Shopify Publication** behind the catalog. This is a one-time setup step per catalog; existing catalogs are not re-provisioned. Ops generally will not need to interact with the Publication directly — it is the backing object that lets the catalog expose products on the storefront for its attached locations.

Data Mapping

NetSuite Field	Shopify Field	Notes
`customer.pricelevel` (child, falling back to parent)	Catalog assignment	Determines which Price List / Catalog a location sees
Company Location IDs (resolved via Shopify API, then integration mappings)	Catalog `companyLocationIds` context	Which locations are attached to each Catalog
Price List ID (from Step 8)	Catalog `priceListId`	Links the Catalog to its Price List
`pricing.unitprice` (per item, per price level)	Fixed product price	Written per product (via its default variant) into the Price List

---

3.10 Orders (Real-Time Webhook)

What it does: When a B2B customer places an order on Shopify, Shopify sends the order payload to the middleware, which translates it into a NetSuite Sales Order and creates it via a NetSuite RESTlet. The job retries up to 3 times on transient failures.

Direction and Frequency: Shopify to NetSuite, real-time (webhook)

Dependency: Customer, location, and product mappings must exist in the integration database for the order's buyer and line items. If a product mapping cannot be found for a line item, that line's NetSuite item internal ID is sent as null and the Sales Order may fail or require manual correction in NetSuite.

Customer Resolution Ladder

The middleware figures out which NetSuite customer to bill in this order of preference:

Attempt	Source	Description
1	`ns_id` note attribute	If the storefront explicitly sets a NetSuite customer ID on the order, it is used as-is
2	`loc_id` note attribute	Resolved to a Company Location mapping, whose NetSuite source ID becomes the customer
3	`order.company.location_id`	The Shopify Company Location on the order, resolved via the integration mapping
4	Shopify customer ID	Looked up as a Company Customer first, then as a Parent Customer, in the integration mapping

Data Mapping

Shopify Field	NetSuite Field	Notes
Resolved customer (see ladder above)	`customerId`	The NetSuite customer the order is billed to
`customer.first_name` + `customer.last_name`	`customerName`	Concatenated, trimmed
`created_at`	`orderDate` and `shipDate`	Formatted as MM/DD/YYYY
`po_number`	`poNumber`	Empty string if not supplied
`note`	`memo`	Falls back to "Thank you for your business!" if blank
`line_items[].title`	line `itemName`	
`line_items[].sku`	line `itemUPC`	
`line_items[].quantity`	line `quantity`	Coerced to integer
`line_items[].product_id` → integration mapping	line `itemInternalId`	Shopify product ID looked up in the mapping table to find the NetSuite item internal ID

Traceability Write-back

After the Sales Order is successfully created in NetSuite, the middleware writes the NetSuite Sales Order ID back onto the Shopify order as a `custom.netsuite_id` metafield. This lets operations jump from a Shopify order to the matching NetSuite Sales Order in one click for audit and lookup.

---

4. Integration Portal Walkthrough

This section is a step-by-step guide to the Arcadian Outfitters Integration Portal — the web application your operations team will use day to day to monitor sync health, investigate errors, trigger manual runs when needed, and manage who receives notifications.

4.1 Logging In

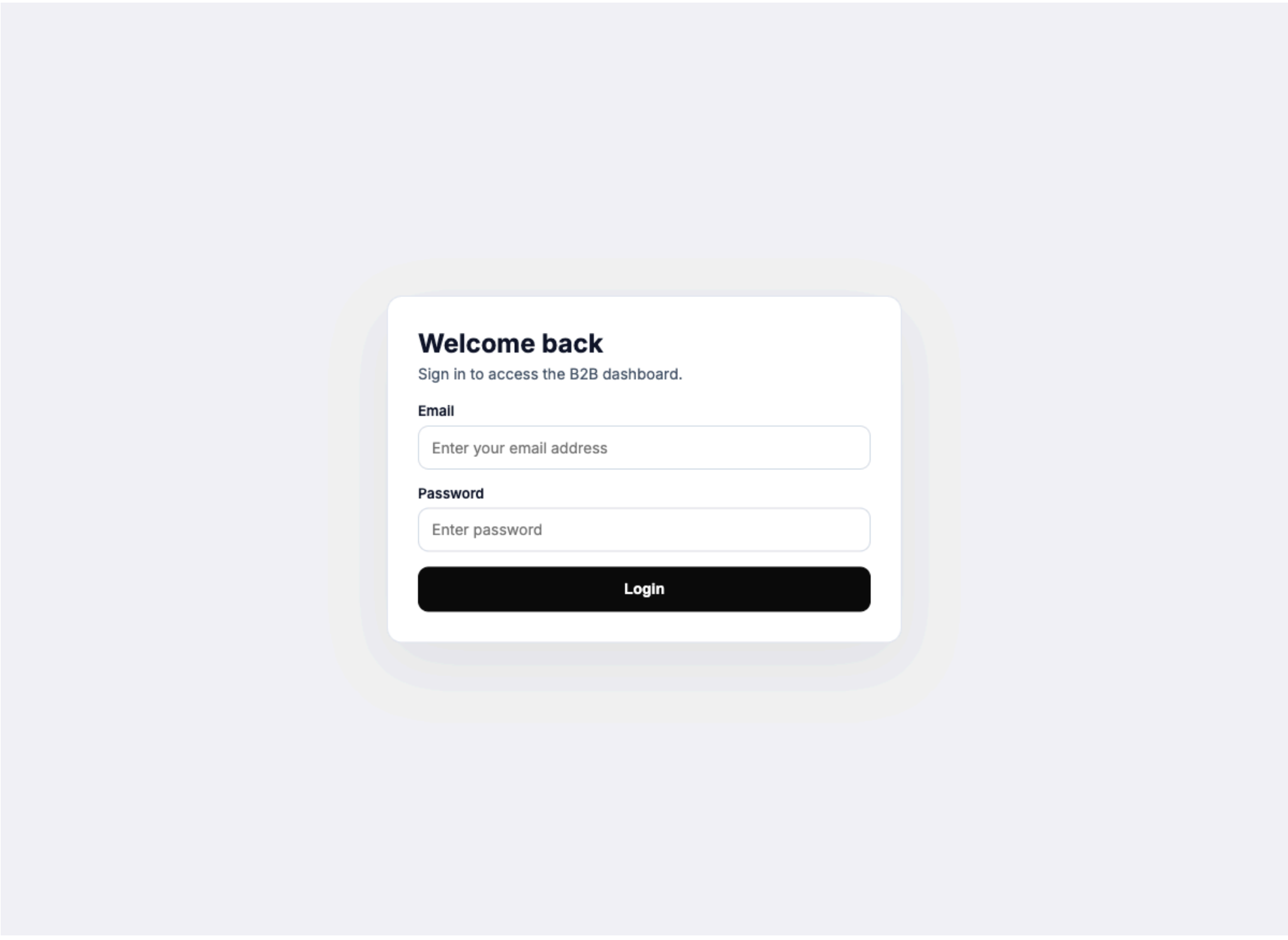
Portal URL: https:///

Login credentials (initial):

Field	Value
Email	`ben@arcadianoutfitters.com`
Password	`913WM34Q88Hi93xaukl5g`

1. Open the URL in your browser. You will see the login screen with the heading **Welcome back**.
2. Enter the **Email** address above in the first field.
3. Enter the **Password** above in the second field.
4. Click the **Login** button.

After a successful login, you are taken directly to the **Overview Dashboard**.



4.2 Dashboard Overview

The Overview Dashboard is your starting point. It gives you a snapshot of sync health across all jobs for a selected time period.

Navigation Sidebar

The left sidebar is visible on every page and contains six items:

- **Overview** — the main dashboard (this page)

- **Jobs** — detailed job listing with manual trigger controls
- **Errors** — log of job runs that encountered failures
- **Schedulers** — view all scheduled sync jobs and their cron configuration
- **Settings** — configure notification email recipients
- **Logout** — sign out of the portal

Date Range Filter

At the top of the dashboard you will see four filter buttons:

Button	Time Range
1 Day	Last 24 hours
3 Days	Last 3 days
7 Days	Last 7 days
14 Days	Last 14 days

Selecting a filter updates the stat cards, the trend chart, and the recent job runs table below.

Stat Cards

Four cards are displayed at the top of the dashboard:

Card	What It Shows
Total Records	The total number of records read from NetSuite across all sync jobs in the selected period.
Success Records	Records that were successfully written, updated, or intentionally skipped. Labeled "Completed without failures."
Failure Records	Records that failed during sync. Labeled "Records with failures."
Success Rate	The percentage of successful records out of total records read (success ÷ total × 100).

> **Why Success Rate can read above 100%.** The formula is `(Written + Updated + Skipped) ÷ Read × 100%`. Customer syncs loop per Shopify location, so a contact attached to multiple locations increments `Skipped` each time it is evaluated, while `Read` counts the source query only once. Treat values at or near 100% as healthy — **Failure Records** is the number to watch.

Records Over Time Chart

Below the stat cards is a line chart titled **Records Over Time**. It plots daily totals across six metrics over the selected date range:

- **Total** (records read)
- **Success** (written + updated + skipped)
- **Written** (new records created in Shopify)
- **Updated** (existing records modified in Shopify)
- **Skipped** (records that required no change)
- **Failed** (records that encountered an error)

Use this chart to spot trends. A sudden spike in the **Failed** line warrants investigation on the Errors page (see Section 4.5).

Recent Job Runs Table

At the bottom of the dashboard is a table showing the most recent run for each sync job. The columns are:

Column	Description
Job	The name of the sync job (e.g., Companies, Products, Inventory).
Last Run	Timestamp of when the job last executed.
Read / Written / Updated / Skipped / Failed	Record counts for the most recent run. See Section 4.4 for what each metric means.
Duration	How long the job took to complete.
Next Run	The scheduled time for the next automatic execution.
Status	A color-coded badge indicating the outcome. See Section 4.4 for status definitions.

- Overview
- Jobs
- Errors
- Schedulers
- Settings
- Logout

Overview Dashboard

1 Day | 3 Days | 7 Days | **14 Days**

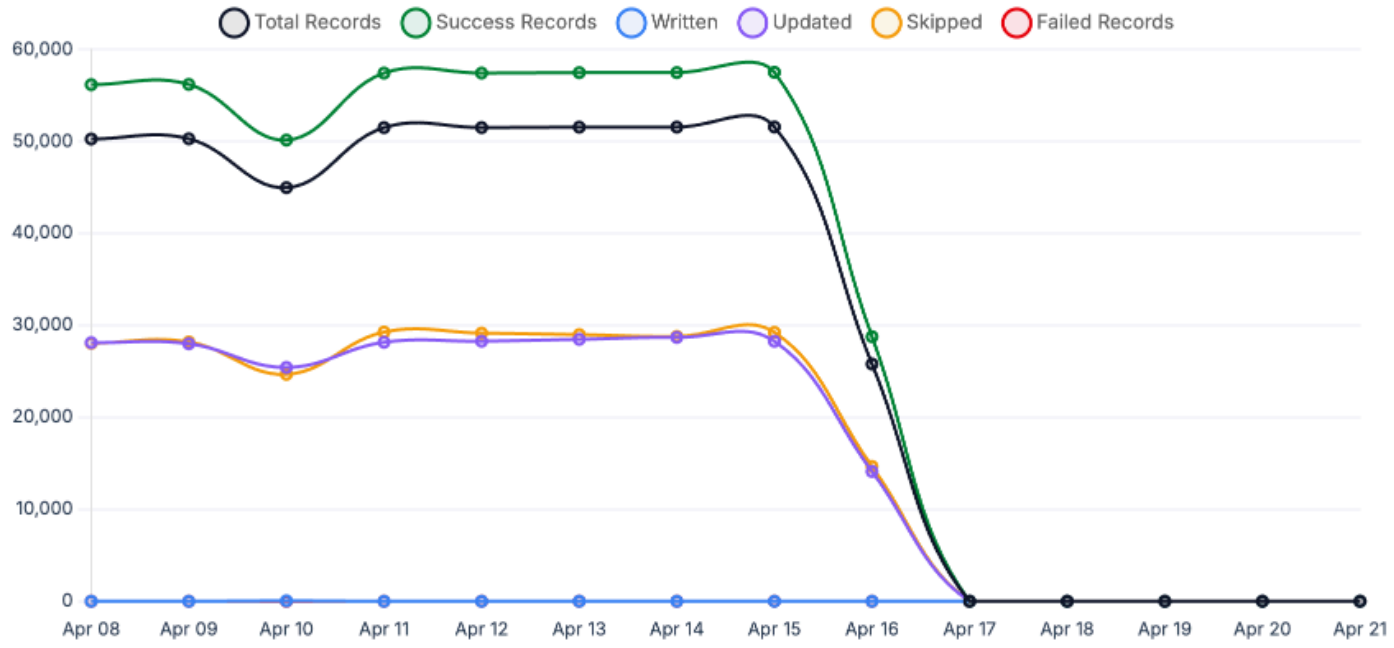
Total Records
479,090
Executions in last 14 day(s)

Success Records
534,605
Completed without failures

Failures Records
32
Records with failures

Success Rate
111.6%
Percentage of successful records

Records Over Time



Recent Job Runs (Last 14 Day(s))

Job	Last Run	Read	Written	Updated	Skipped	Failed	Duration	Next Run	Status	Actions
Catalogs	2026-04-16 11:10:49	9	0	3	6	0	1m 16s	2026-04-16 15:10:49	Success	
PriceLists	2026-04-16 11:10:47	4	0	3	1	0	0m 2s	2026-04-16 15:10:47	Success	
Inventory	2026-04-16 10:58:22	2,711	0	0	2,711	0	12m 25s	2026-04-16 11:58:22	Success	
Products	2026-04-16 10:26:17	762	0	757	5	0	32m 5s	2026-04-16 14:26:19	Success	
InventoryLocation	2026-04-16 10:26:15	3	0	3	0	0	0m 2s	2026-04-16 14:26:16	Success	
Customers	2026-04-16 10:10:32	718	0	582	880	0	15m 43s	2026-04-16 14:10:32	Success	
Locations	2026-04-16 09:30:28	653	0	653	0	0	40m 4s	2026-04-16 13:30:28	Success	
ParentCustomers	2026-04-16 09:07:07	813	0	741	72	0	23m 21s	2026-04-16 13:07:07	Success	
Companies	2026-04-16 09:00:04	774	0	774	0	0	7m 3s	2026-04-16 13:00:04	Success	

4.3 Triggering a Manual Sync

Under normal operation all sync jobs run automatically on a schedule (see Section 4.6). There are, however, situations where you may want to run a job immediately — for example, after fixing a record in NetSuite that you want reflected in Shopify right away, or after a failed run where you have confirmed the underlying issue is resolved.

How to Trigger a Manual Sync

1. Click **Jobs** in the sidebar to open the **Flow Health by Job** table.
2. Locate the job you want to run.
3. In the right-hand action area of that row, click the **play icon**. This sends a `POST` request to `/job/run/{jobName}`, which queues the job for immediate execution on the server.
4. A success banner appears at the top of the page (for example, *"Products job triggered successfully!"*).
5. The job's metrics and status refresh once the run completes.

Available Jobs

The play icon is available on the following rows:

Job Name	What It Syncs
Companies	Company (parent customer) records from NetSuite to Shopify
ParentCustomers	Parent customer records in NetSuite
Locations	Company location data
Customers	Customer (contact) records linked to company locations
InventoryLocation	Warehouse locations used for inventory tracking
Products	Product catalog (simple products, no variants)
Inventory	Stock levels at each warehouse
PriceLists	Pricing information
Catalogs	Catalog assignments linking companies to price lists
Orders	Orders are populated automatically via real-time webhook (no manual trigger needed — see Section 4.4)

Sync Dependency Order — Read This Before Triggering

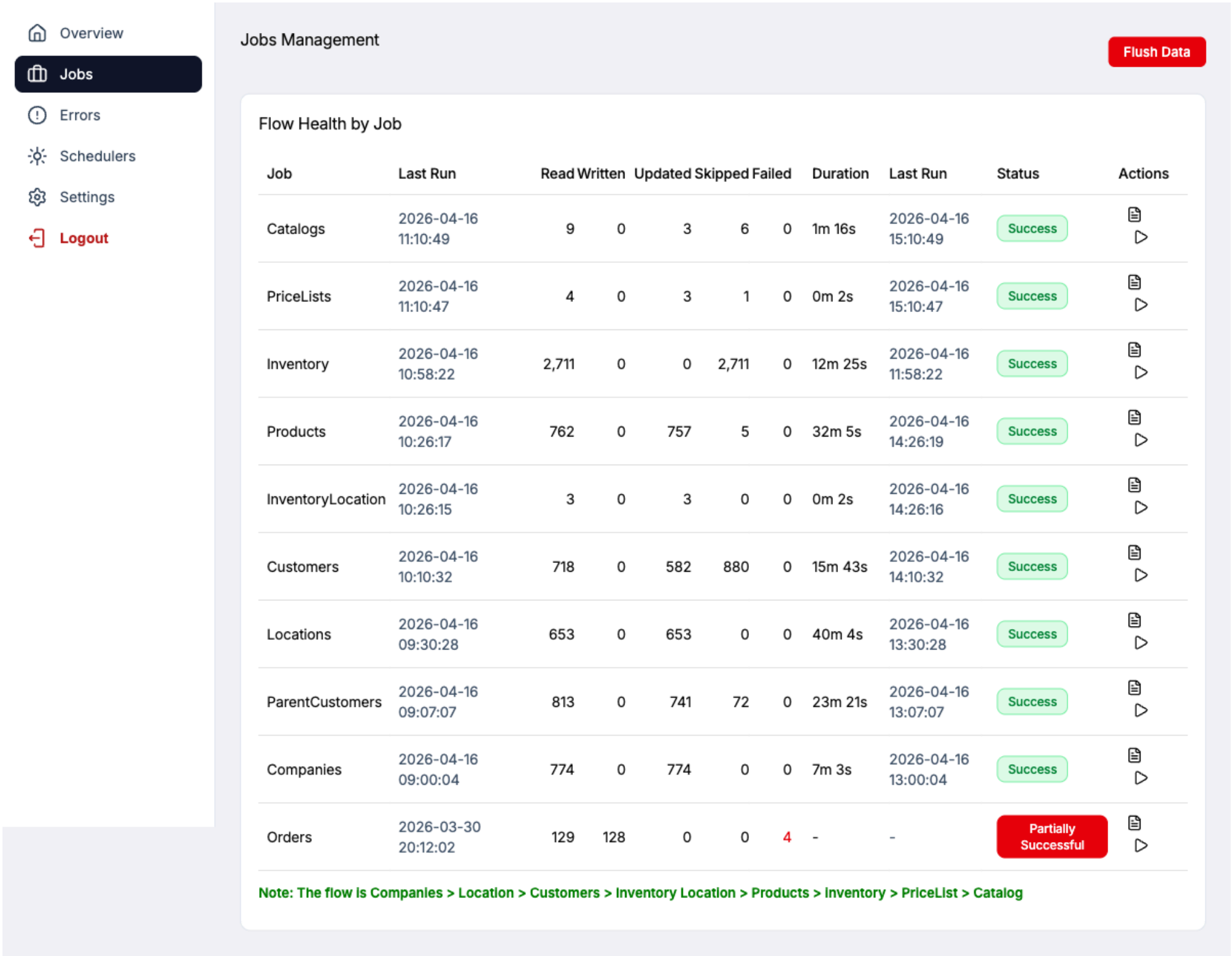
Sync jobs depend on each other. Data created by one job is referenced by jobs that run after it. If you trigger jobs out of order, you may cause failures because the records a job depends on do not yet exist.

The dependency chain is also printed at the bottom of the Jobs page:

> *"Note: The flow is Companies > Location > Customers > Inventory Location > Products > inventory > PriceList > Catalog"*

4.4 Checking Job Status / Flow Health

The **Jobs** page is where you confirm each job's current health at a glance.



Metric Columns

Each row breaks down what happened during the most recent run:

Metric	Definition
Last Run	Timestamp of the most recent run.
Read	Number of records fetched from NetSuite.
Written	New records created in Shopify. These did not previously exist.
Updated	Existing Shopify records modified because the source data had changed.
Skipped	Records that required no change (source and destination already matched).
Failed	Records that encountered an error. See Section 4.5 for how to investigate.
Duration	How long the run took, start to finish.
Next Run	The scheduled time for the next automatic execution.
Status	A color-coded badge — see below.

Status Badges

Status	Badge Color	Meaning
Success	Green	All records processed successfully. Zero failures.
Partially Successful	Orange	Some records succeeded and some failed. This is the most common non-success state. Typical causes are API rate limiting, NetSuite concurrency limits, or transient network errors. Failures in a partially successful run will usually succeed on the next scheduled run.
Failed	Red	All records failed, or a critical error prevented the job from completing.

> **What to do about "Partially Successful":** In most cases, no action is needed. The affected records retry on the next scheduled cycle (within three hours) and typically succeed. Escalate only when the same records continue to fail across multiple consecutive runs (see Section 4.5).

Special Case — the Orders Row

Orders are handled differently from every other job in the list.

- There is **no scheduled sync** for orders.
- Orders are created in NetSuite via a **real-time webhook** fired by Shopify whenever a customer places an order on the storefront.
- The Orders row on the Jobs page is therefore a **rolling counter of webhook activity**, not a record of a scheduled run. The Last Run, Read, Written, and Failed values reflect the most recent order events processed.

In practice this means you should use the Orders row to confirm that orders are flowing, but you do not need to — and cannot usefully — trigger it manually.

4.5 Investigating Errors

When a job finishes with a **Partially Successful** or **Failed** status, you will want to find out which records were affected and why.

Two Ways In

Entry Point	What You See
Errors in the sidebar	The **Logs Management** page — every job run (across all jobs) that had at least one failure.
Log icon next to a row on the **Jobs** page	The failed records from **that job's most recent run** only.

Use the Errors page when you want to triage across the whole integration. Use the per-row log icon when you already know which job you are investigating.

Logs Management

All Failed Runs

Job	Last Run	Read	Written	Updated	Skipped	Failed	Duration	Status
Locations	2026-04-16 06:30:23	653	0	652	0	1	59s	Partially Successful
Products	2026-04-15 16:32:00	762	0	756	5	1	16s	Partially Successful
Products	2026-04-15 10:27:27	763	0	757	5	1	8s	Partially Successful
Products	2026-04-15 01:30:47	763	0	757	5	1	3s	Partially Successful
Products	2026-04-14 22:32:03	763	0	757	5	1	27s	Partially Successful
Locations	2026-04-14 21:32:43	652	0	651	0	1	59s	Partially Successful
Locations	2026-04-14 06:30:41	653	0	651	0	2	27s	Partially Successful
Products	2026-04-14 04:24:06	763	0	757	5	1	13s	Partially Successful
Products	2026-04-14 01:28:12	763	0	757	5	1	5s	Partially Successful
Products	2026-04-13 22:29:55	763	0	757	5	1	49s	Partially Successful
Companies	2026-04-13 18:00:02	780	0	779	0	1	33s	Partially Successful
Locations	2026-04-13 15:32:50	653	0	652	0	1	13s	Partially Successful
Products	2026-04-13 10:23:31	760	0	753	5	2	39s	Partially Successful
Products	2026-04-13 04:24:25	760	0	754	5	1	48s	Partially Successful
Locations	2026-04-13 03:30:09	653	0	652	0	1	54s	Partially Successful
Products	2026-04-13 01:26:13	760	0	754	5	1	29s	Partially Successful
Locations	2026-04-12 18:30:32	653	0	652	0	1	18s	Partially Successful
Locations	2026-04-12 09:28:47	653	0	652	0	1	22s	Partially Successful
Products	2026-04-12 01:23:57	760	0	753	5	2	8s	Partially Successful
Locations	2026-04-11 21:29:55	653	0	652	0	1	5s	Partially Successful
Products	2026-04-11 07:18:13	760	0	754	5	1	48s	Partially Successful
Companies	2026-04-10 15:00:02	784	0	783	0	1	2s	Partially Successful
Companies	2026-04-10 12:00:03	784	0	783	0	1	3s	Partially Successful
Products	2026-04-10 01:27:38	726	0	721	4	1	33s	Partially Successful
Locations	2026-04-10 00:31:13	648	0	647	0	1	30s	Partially Successful
Products	2026-04-09 22:28:50	726	0	721	4	1	10s	Partially Successful

Locations	2026-04-09 15:32:45	648	0	647	0	1	30s	Partially Successful
Products	2026-04-08 07:23:34	726	0	723	2	1	51s	Partially Successful
Products	2026-04-07 13:28:23	726	0	723	2	1	45s	Partially Successful
Locations	2026-04-06 21:30:46	647	0	646	0	1	25s	Partially Successful
Locations	2026-04-06 18:31:27	647	0	646	0	1	15s	Partially Successful
Products	2026-04-06 10:20:03	723	0	716	2	5	20s	Partially Successful
Locations	2026-04-06 00:30:02	647	0	646	0	1	35s	Partially Successful
Products	2026-04-04 22:25:10	723	0	720	2	1	47s	Partially Successful
Products	2026-04-03 16:24:59	719	0	716	2	1	20s	Partially Successful
Companies	2026-04-03 15:00:03	785	0	784	0	1	43s	Partially Successful
Companies	2026-04-01 12:00:03	1,354	0	1,346	8	1	44s	Partially Successful
Locations	2026-04-01 09:32:15	690	0	680	9	1	29s	Partially Successful
Companies	2026-04-01 09:00:02	1,354	0	1,346	8	1	45s	Partially Successful
Products	2026-04-01 07:08:01	719	0	715	3	1	29s	Partially Successful
Companies	2026-04-01 06:00:02	1,354	0	1,346	8	1	56s	Partially Successful
Companies	2026-04-01 03:00:03	1,354	0	1,346	8	1	24s	Partially Successful
Companies	2026-04-01 00:00:02	1,354	0	1,346	8	1	37s	Partially Successful
Locations	2026-03-31 21:33:01	690	0	680	9	1	24s	Partially Successful
Companies	2026-03-31 21:00:02	1,354	0	1,346	8	1	31s	Partially Successful
Companies	2026-03-31 18:00:03	1,354	0	1,346	8	1	11s	Partially Successful
Companies	2026-03-31 15:00:03	1,354	0	1,346	8	1	14s	Partially Successful
Companies	2026-03-31 12:00:02	1,354	0	1,346	8	1	42s	Partially Successful
Inventory	2026-03-31 11:07:42	0	0	0	0	0	45s	Running
Products	2026-03-31 10:25:58	716	0	712	3	1	44s	Partially Successful
Companies	2026-03-31 09:00:03	1,354	0	1,346	8	1	38s	Partially Successful
Companies	2026-03-31 06:00:03	1,354	0	1,346	8	1	29s	Partially Successful
Companies	2026-03-31 03:00:03	1,354	0	1,346	8	1	25s	Partially Successful
Companies	2026-03-31 00:00:03	1,354	0	1,346	8	1	17s	Partially Successful

Companies	2026-03-30 21:00:02	1,354	1	1,345	8	1	41s	Partially Successful
Orders	2026-03-30 20:12:02	129	128	0	0	4	34s	Partially Successful
Companies	2026-03-30 18:00:02	1,353	0	1,345	8	1	6s	Partially Successful
Customers	2026-03-30 15:58:22	422	0	370	52	1	59s	Partially Successful
Companies	2026-03-30 15:00:03	1,353	0	1,345	8	1	12s	Partially Successful
Companies	2026-03-30 12:00:02	1,353	1	1,344	8	1	47s	Partially Successful
Products	2026-03-29 18:56:38	717	0	713	3	1	56s	Partially Successful
Products	2026-03-27 21:56:45	717	0	713	3	1	23s	Partially Successful
ParentCustomers	2026-03-26 15:00:02	250	0	242	8	2	20s	Partially Successful
ParentCustomers	2026-03-26 12:00:04	250	0	242	8	2	35s	Partially Successful
ParentCustomers	2026-03-26 06:00:03	250	0	242	8	2	58s	Partially Successful
ParentCustomers	2026-03-26 03:00:02	250	0	242	8	2	56s	Partially Successful
ParentCustomers	2026-03-26 00:00:02	250	0	242	8	2	13s	Partially Successful
ParentCustomers	2026-03-25 21:00:02	250	0	242	8	2	5s	Partially Successful
Products	2026-03-25 18:59:35	709	0	705	3	1	0s	Partially Successful
ParentCustomers	2026-03-25 18:00:02	250	0	242	8	2	55s	Partially Successful
ParentCustomers	2026-03-25 15:00:02	250	0	242	8	2	11s	Partially Successful
ParentCustomers	2026-03-25 12:00:02	250	0	242	8	2	39s	Partially Successful
ParentCustomers	2026-03-25 09:00:02	250	0	242	8	2	58s	Partially Successful
Customers	2026-03-25 06:41:33	422	0	370	52	1	29s	Partially Successful
ParentCustomers	2026-03-25 06:00:02	250	0	242	8	2	11s	Partially Successful
ParentCustomers	2026-03-25 03:00:02	250	0	242	8	2	44s	Partially Successful
ParentCustomers	2026-03-25 00:00:03	250	0	242	8	2	59s	Partially Successful
ParentCustomers	2026-03-24 21:00:03	250	0	242	8	2	53s	Partially Successful
Products	2026-03-24 19:05:57	709	0	706	2	1	24s	Partially Successful
ParentCustomers	2026-03-24 18:00:02	250	0	242	8	2	48s	Partially Successful
ParentCustomers	2026-03-24 15:00:03	250	0	242	8	2	23s	Partially Successful
ParentCustomers	2026-03-24 12:00:02	250	0	242	8	2	22s	Partially Successful
ParentCustomers	2026-03-24 09:00:02	250	0	242	8	2	25s	Partially Successful

ParentCustomers	2026-03-24 05:05:02	250	0	242	8	2	54s	Partially Successful
ParentCustomers	2026-03-24 04:05:02	250	0	242	8	2	43s	Partially Successful
ParentCustomers	2026-03-24 03:05:02	250	0	242	8	2	34s	Partially Successful
ParentCustomers	2026-03-24 02:05:02	250	0	242	8	2	44s	Partially Successful
ParentCustomers	2026-03-24 01:05:02	250	0	242	8	2	19s	Partially Successful
Products	2026-03-24 01:04:28	709	0	706	2	1	42s	Partially Successful
ParentCustomers	2026-03-24 00:05:02	250	0	242	8	2	26s	Partially Successful
Products	2026-03-24 00:02:29	709	0	706	2	1	15s	Partially Successful
ParentCustomers	2026-03-23 23:05:01	250	0	242	8	2	1s	Partially Successful
Locations	2026-03-23 22:27:28	685	0	679	5	1	48s	Partially Successful
ParentCustomers	2026-03-23 22:05:03	250	0	242	8	2	24s	Partially Successful
ParentCustomers	2026-03-23 21:05:03	250	0	242	8	2	11s	Partially Successful
ParentCustomers	2026-03-23 20:05:03	250	0	242	8	2	44s	Partially Successful
ParentCustomers	2026-03-23 19:05:02	250	0	242	8	2	23s	Partially Successful
ParentCustomers	2026-03-23 18:05:02	250	0	242	8	2	43s	Partially Successful
ParentCustomers	2026-03-23 17:05:02	250	0	242	8	2	34s	Partially Successful
ParentCustomers	2026-03-23 16:05:02	250	0	242	8	2	34s	Partially Successful
ParentCustomers	2026-03-23 15:05:03	250	0	242	8	2	59s	Partially Successful
ParentCustomers	2026-03-23 14:05:01	250	0	242	8	2	5s	Partially Successful
ParentCustomers	2026-03-23 13:05:02	250	0	242	8	2	24s	Partially Successful
ParentCustomers	2026-03-23 12:05:02	250	0	242	8	2	27s	Partially Successful
ParentCustomers	2026-03-23 11:05:03	250	0	242	8	2	30s	Partially Successful
ParentCustomers	2026-03-23 09:05:02	250	0	242	8	2	27s	Partially Successful
ParentCustomers	2026-03-23 08:05:01	250	0	242	8	2	11s	Partially Successful
ParentCustomers	2026-03-23 07:05:02	250	0	242	8	2	44s	Partially Successful
ParentCustomers	2026-03-23 06:05:02	250	0	242	8	2	8s	Partially Successful
ParentCustomers	2026-03-23 05:05:02	250	0	242	8	2	46s	Partially Successful
ParentCustomers	2026-03-23 04:05:02	250	0	242	8	2	28s	Partially Successful

Locations	2026-03-23 03:25:41	684	0	678	5	1	21s	Partially Successful
ParentCustomers	2026-03-23 03:05:02	250	0	242	8	2	39s	Partially Successful
ParentCustomers	2026-03-23 02:05:05	250	0	242	8	4	20s	Partially Successful
ParentCustomers	2026-03-23 01:05:02	250	0	242	8	2	16s	Partially Successful
ParentCustomers	2026-03-23 00:05:02	250	0	242	8	2	0s	Partially Successful
ParentCustomers	2026-03-22 23:05:02	250	0	242	8	2	56s	Partially Successful
ParentCustomers	2026-03-22 22:05:03	250	0	242	8	2	13s	Partially Successful
ParentCustomers	2026-03-22 21:05:02	250	0	242	8	2	18s	Partially Successful
Locations	2026-03-22 20:26:55	684	0	678	5	1	51s	Partially Successful
ParentCustomers	2026-03-22 20:05:02	250	0	242	8	2	52s	Partially Successful
ParentCustomers	2026-03-22 19:05:02	250	0	242	8	2	18s	Partially Successful
Products	2026-03-22 19:00:37	708	0	705	2	1	13s	Partially Successful
Locations	2026-03-22 18:25:46	684	0	679	5	1	55s	Partially Successful
ParentCustomers	2026-03-22 18:05:02	250	0	242	8	2	42s	Partially Successful
ParentCustomers	2026-03-22 17:05:01	250	0	242	8	2	26s	Partially Successful
ParentCustomers	2026-03-22 16:05:02	250	0	242	8	2	1s	Partially Successful
ParentCustomers	2026-03-22 15:05:02	250	0	242	8	2	48s	Partially Successful
ParentCustomers	2026-03-22 14:05:02	250	0	242	8	2	8s	Partially Successful
ParentCustomers	2026-03-22 13:05:01	250	0	242	8	2	52s	Partially Successful
ParentCustomers	2026-03-22 12:05:03	250	0	242	8	2	49s	Partially Successful
Products	2026-03-22 12:00:56	708	0	705	2	1	16s	Partially Successful
ParentCustomers	2026-03-22 11:05:02	250	0	242	8	2	28s	Partially Successful
ParentCustomers	2026-03-22 10:05:03	250	0	242	8	2	15s	Partially Successful
ParentCustomers	2026-03-22 09:05:02	250	0	242	8	2	36s	Partially Successful
ParentCustomers	2026-03-22 08:05:01	250	0	242	8	2	30s	Partially Successful
ParentCustomers	2026-03-22 07:05:02	250	0	242	8	2	57s	Partially Successful
ParentCustomers	2026-03-22 06:05:02	250	0	242	8	2	8s	Partially Successful
Products	2026-03-22 05:05:34	708	0	705	2	1	3s	Partially Successful

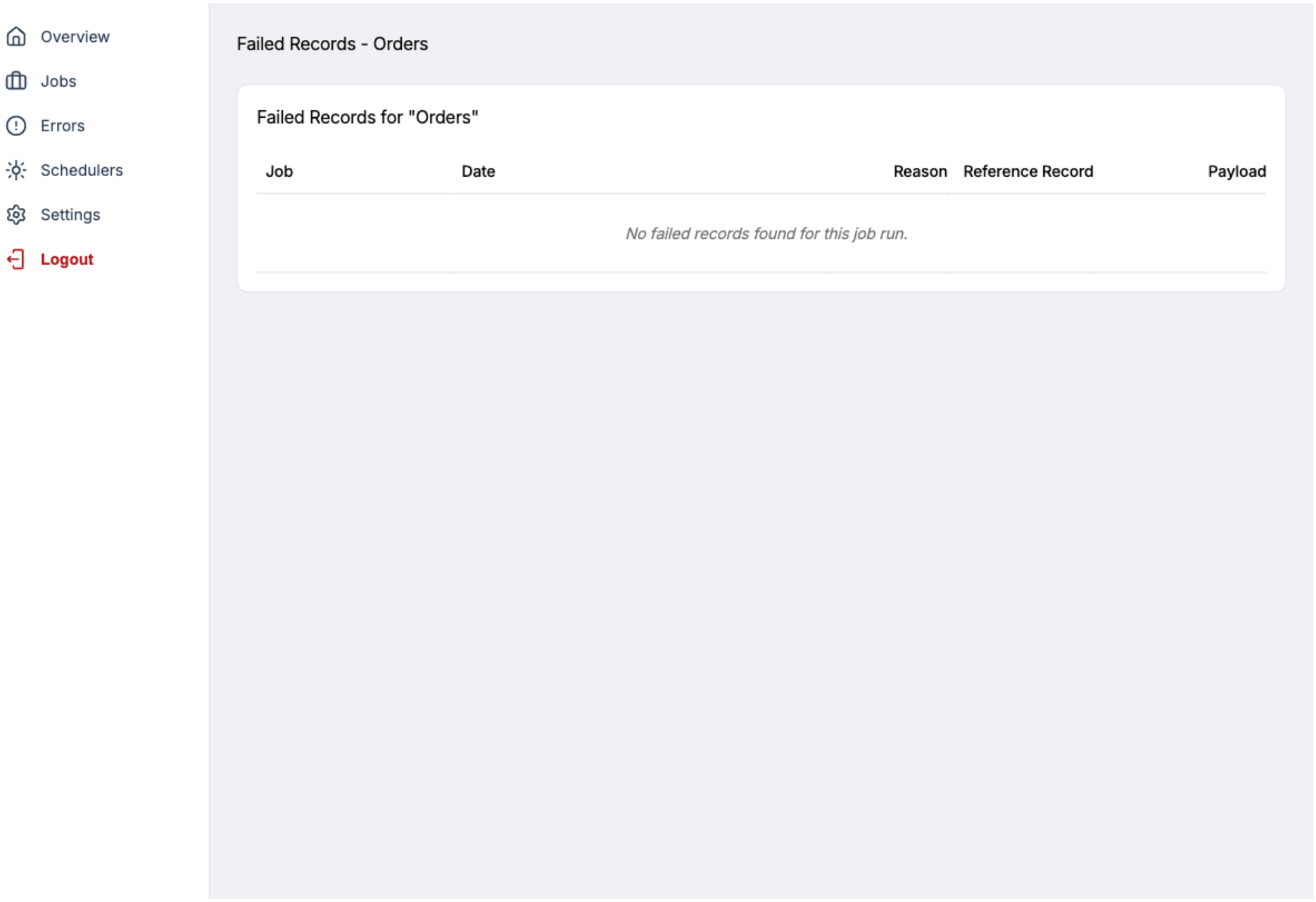
ParentCustomers	2026-03-22 05:05:02	250	0	242	8	2	23s	Partially Successful
ParentCustomers	2026-03-22 04:05:02	250	0	242	8	2	9s	Partially Successful
Products	2026-03-22 03:06:23	708	0	705	2	1	56s	Partially Successful
ParentCustomers	2026-03-22 03:05:02	250	0	242	8	2	59s	Partially Successful
ParentCustomers	2026-03-22 02:05:03	250	0	242	8	2	49s	Partially Successful
ParentCustomers	2026-03-22 01:05:03	250	0	242	8	2	35s	Partially Successful
ParentCustomers	2026-03-22 00:05:02	250	0	242	8	2	47s	Partially Successful
ParentCustomers	2026-03-21 23:05:02	250	0	242	8	2	6s	Partially Successful
ParentCustomers	2026-03-21 22:05:02	250	0	242	8	2	8s	Partially Successful
ParentCustomers	2026-03-21 21:05:03	250	0	242	8	2	42s	Partially Successful
ParentCustomers	2026-03-21 20:05:02	250	0	242	8	2	24s	Partially Successful
ParentCustomers	2026-03-21 19:05:02	250	0	242	8	2	0s	Partially Successful
ParentCustomers	2026-03-21 18:05:02	250	0	242	8	2	11s	Partially Successful
ParentCustomers	2026-03-21 17:05:03	250	0	242	8	2	3s	Partially Successful
Locations	2026-03-21 16:27:00	684	0	678	5	1	34s	Partially Successful
ParentCustomers	2026-03-21 16:05:02	250	0	242	8	2	58s	Partially Successful
ParentCustomers	2026-03-21 15:05:02	250	0	242	8	2	56s	Partially Successful
ParentCustomers	2026-03-21 14:05:02	250	0	242	8	2	34s	Partially Successful
ParentCustomers	2026-03-21 13:05:02	250	0	242	8	2	13s	Partially Successful
Locations	2026-03-21 12:27:09	684	0	677	5	2	11s	Partially Successful
ParentCustomers	2026-03-21 12:05:02	250	0	242	8	2	6s	Partially Successful
ParentCustomers	2026-03-21 11:05:02	250	0	242	8	2	18s	Partially Successful
ParentCustomers	2026-03-21 10:05:03	250	0	242	8	2	28s	Partially Successful
ParentCustomers	2026-03-21 09:05:02	250	0	242	8	2	11s	Partially Successful
ParentCustomers	2026-03-21 08:05:02	250	0	242	8	2	30s	Partially Successful
ParentCustomers	2026-03-21 07:05:02	250	0	242	8	2	29s	Partially Successful
Locations	2026-03-21 06:27:37	684	0	678	5	1	35s	Partially Successful
ParentCustomers	2026-03-21 06:05:02	250	0	242	8	2	35s	Partially Successful
ParentCustomers	2026-03-21 05:05:02	250	0	242	8	2	41s	Partially Successful

ParentCustomers	2026-03-21 04:05:02	250	0	242	8	2	57s	Partially Successful
ParentCustomers	2026-03-21 03:05:02	250	0	242	8	2	7s	Partially Successful
ParentCustomers	2026-03-21 02:05:02	250	0	242	8	2	10s	Partially Successful
ParentCustomers	2026-03-21 01:05:02	250	0	242	8	2	59s	Partially Successful
ParentCustomers	2026-03-21 00:05:02	250	0	242	8	2	3s	Partially Successful
Products	2026-03-20 23:59:57	708	0	705	2	1	26s	Partially Successful
ParentCustomers	2026-03-20 23:05:03	250	0	242	8	2	16s	Partially Successful
ParentCustomers	2026-03-20 22:05:02	250	0	242	8	2	36s	Partially Successful
Locations	2026-03-20 21:27:28	684	0	678	5	1	12s	Partially Successful
Products	2026-03-20 21:06:44	708	0	705	2	1	31s	Partially Successful
ParentCustomers	2026-03-20 21:05:02	250	0	242	8	2	26s	Partially Successful
ParentCustomers	2026-03-20 20:05:02	250	0	242	8	2	19s	Partially Successful
Products	2026-03-20 19:09:05	708	0	705	2	1	1s	Partially Successful
ParentCustomers	2026-03-20 19:05:02	250	0	242	8	3	39s	Partially Successful
ParentCustomers	2026-03-20 18:05:02	250	0	242	8	2	23s	Partially Successful
ParentCustomers	2026-03-20 17:05:03	250	0	242	8	2	40s	Partially Successful
ParentCustomers	2026-03-20 16:05:02	250	0	242	8	2	4s	Partially Successful
ParentCustomers	2026-03-20 15:05:02	250	0	242	8	2	13s	Partially Successful
ParentCustomers	2026-03-20 14:03:07	250	0	242	8	2	17s	Partially Successful
ParentCustomers	2026-03-19 04:59:56	250	242	0	8	2	37s	Partially Successful

Reading the Failure Detail Page

Clicking either entry point opens the **Failed Records** view for a specific job run. Each failed record is listed with the following columns:

Column	Description
Job	The name of the sync job.
Date	When the failure occurred.
Reason	A short description of what went wrong (for example, API timeout, validation error, rate limit exceeded).
Reference Record	The identifier of the specific record that failed — typically a NetSuite internal ID. Use this to locate the record in NetSuite or Shopify.
Payload	The full data that was being processed at the moment of failure. Useful for reproducing the error.



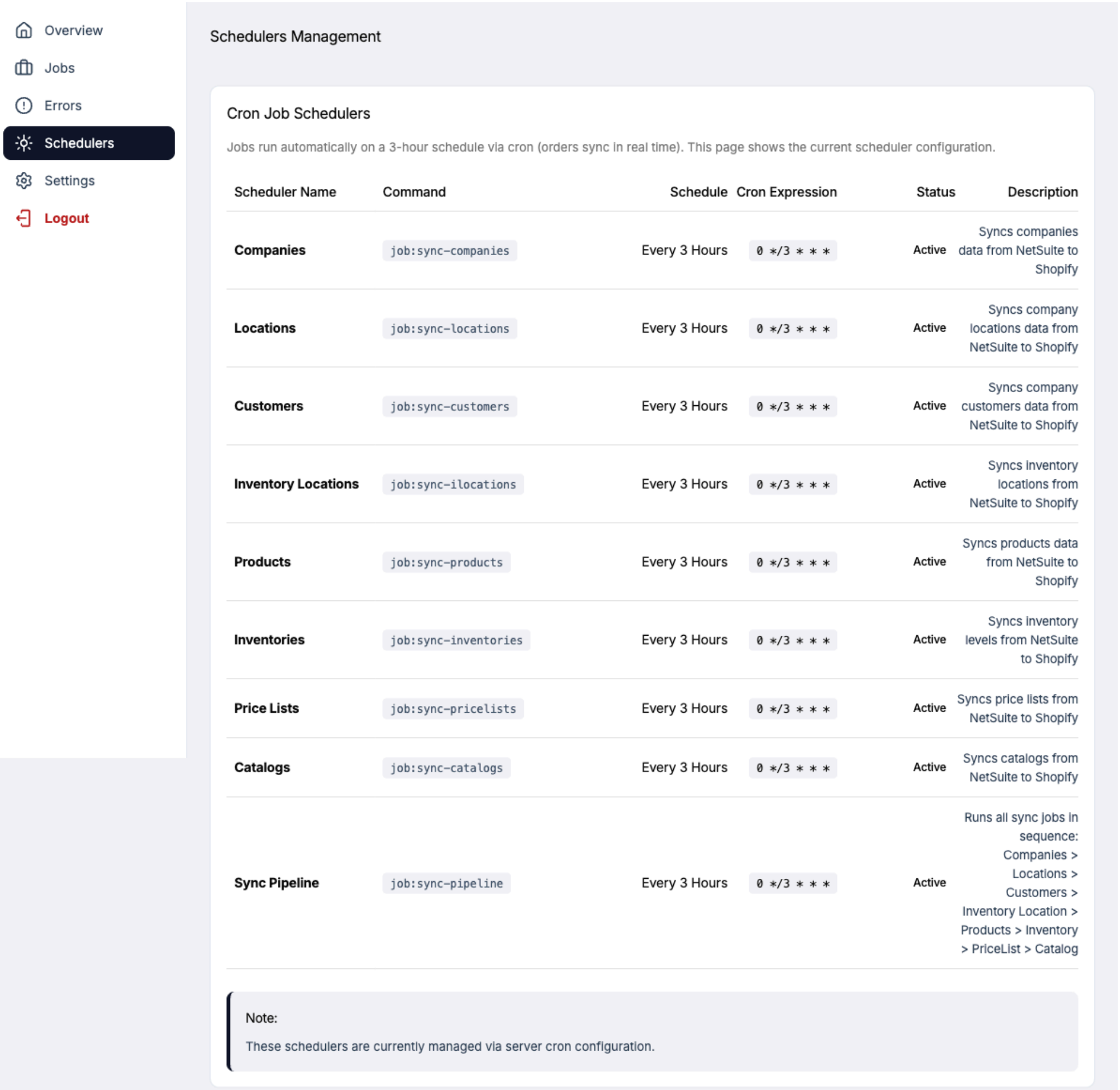
> **If you need to send a failure to Makro Agency:** include the **Job**, the **Date**, the **Reason** text, the **Reference Record** ID, and a copy of the **Payload**. That combination is everything we need to diagnose the issue without asking follow-up questions.

Decide Whether to Act

Scenario	Action
A small number of records failed once with a rate-limit or timeout reason.	**No action needed.** These will retry automatically on the next scheduled cycle (within three hours).
The same records fail across 2–3 consecutive runs.	Review the reason and payload. Check whether the source record in NetSuite has bad or missing data.
The same records fail across 4 or more consecutive runs.	**Escalate to Makro Agency.** Send the job name, reference IDs, reasons, and one sample payload.
An entire job fails with a critical error.	**Escalate to Makro Agency immediately.** This usually indicates a system-level issue.

4.6 Schedulers

The **Schedulers** page lists every configured sync job and the command, cron expression, and status for each one.



What Each Row Corresponds To

Scheduler	Artisan Command	What It Does
Companies	`job:sync-companies`	Syncs parent company records from NetSuite to Shopify
Locations	`job:sync-locations`	Syncs company locations
Customers	`job:sync-customers`	Syncs customer contacts attached to each company location
Inventory Locations	`job:sync-ilocations`	Syncs warehouse locations used for inventory tracking
Products	`job:sync-products`	Syncs products (simple products only — Arcadian does not use variants)
Inventories	`job:sync-inventories`	Syncs inventory levels at each warehouse
Price Lists	`job:sync-pricelists`	Syncs B2B price lists
Catalogs	`job:sync-catalogs`	Syncs catalog assignments linking companies to price lists
Sync Pipeline	`job:sync-pipeline`	Runs all of the above, in the correct dependency order, as a single sequence

> **Parent Customers is missing from this view.** The Schedulers page does not show a dedicated row for the Parent Customers sync (`job:sync-pcustomers`), but Parent Customers **is** included in the pipeline and runs as Step 2 of every scheduled cycle. This is a known gap in the Schedulers page display; it does not affect the sync itself.

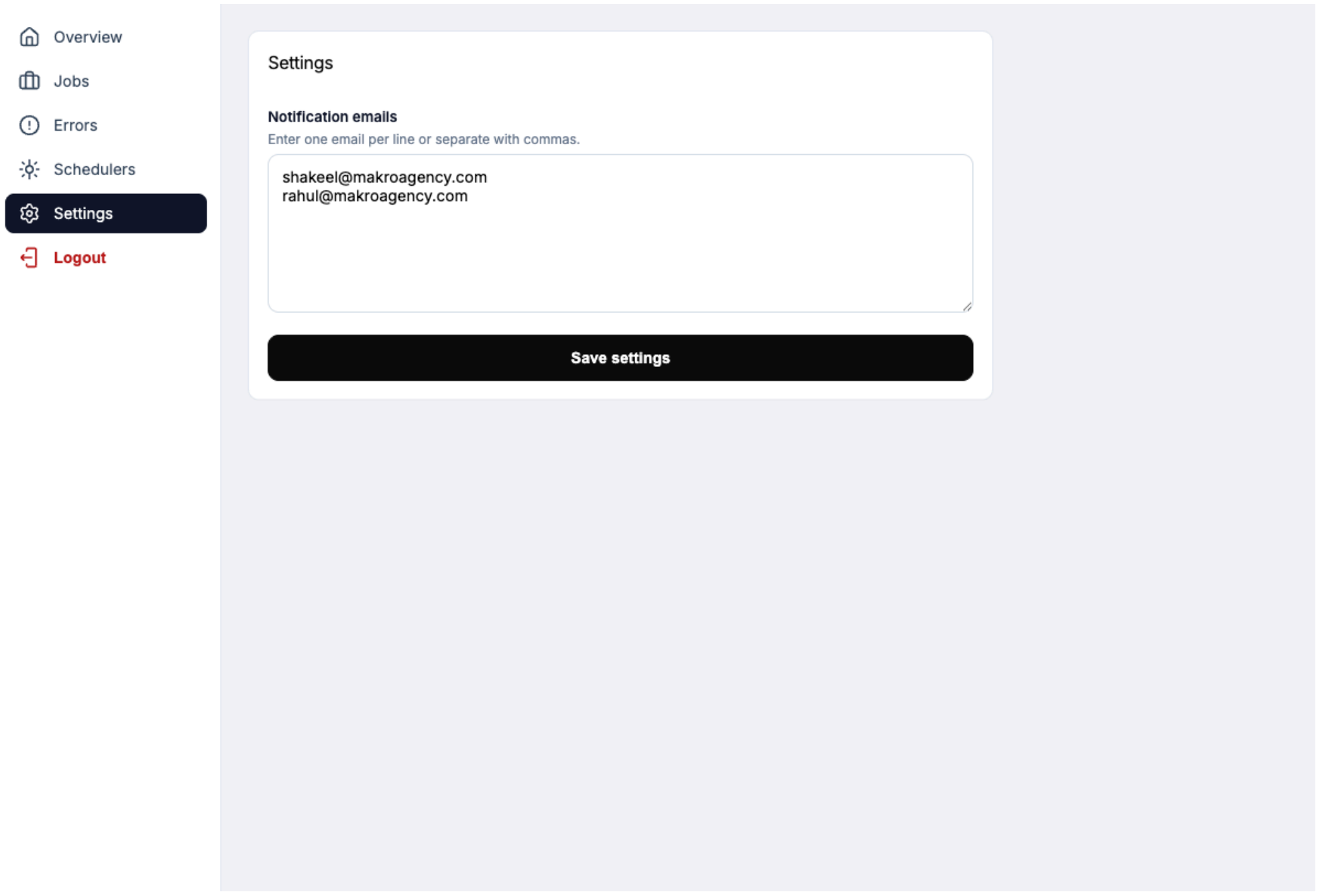
The footer confirms: *"These schedulers are currently managed via server cron configuration."* The cron expression `0 */3 * * *` shown next to each row is the one that governs the production schedule — it runs the pipeline at minute 0 of every third hour.

Changing the Schedule

Schedules cannot be edited through the portal. If you need the cadence adjusted (for example, to slow the pipeline down during a quiet period or to rerun more often during a migration window), contact Makro Agency.

4.7 Settings and Notifications

The **Settings** page controls who receives email notifications from the integration.



Adding or Removing Recipients

1. Click **Settings** in the sidebar.
2. Under the **Notification emails** heading you will see a textarea pre-filled with the current list.
3. Edit the list. You can use either format:
   - **One email per line**
   - **Comma-separated on a single line**
4. Click **Save settings**. The page reloads with the updated list in place.

To remove someone, delete their email address from the textarea and save.

Who Receives What

Every address in this textarea receives **failure notifications only** — the integration does not send routine pipeline-start or pipeline-complete emails, so the inbox stays quiet unless something needs attention.

Notification Type	When It Is Sent	Triggered By
Per-job failure	When an individual sync job encounters failed records	`JobFailureNotifier`

> **Recommendation:** Maintain at least two active recipients at all times, so that failure alerts are never missed when one person is out of office. Review the list periodically (for example, during quarterly IT reviews) to remove anyone who has left the team.

---

5. Frequently Asked Questions

How often does data sync?

The full sync pipeline runs **every three hours**, automatically, via the server cron. The schedule is `0 */3 * * *` — it begins at minute 0 of every third hour (00:00, 03:00, 06:00, 09:00, 12:00, 15:00, 18:00, and 21:00). Each run executes the nine jobs in sequence (Companies through Catalogs). Real-time order creation from Shopify to NetSuite runs independently of this schedule and fires the moment a customer places an order.

A sync "failed" on the dashboard — what do I do?

Most failures are transient — Shopify or NetSuite rate limits, network timeouts, or NetSuite concurrency locks. The next scheduled pipeline usually resolves them without intervention. Go to the **Jobs** page, open the failed job to review the error payload on the Errors page, and if the same issue persists across three or more consecutive runs, email Makro Agency with that payload attached.

I changed a customer or product in NetSuite — when will it appear in Shopify?

It will be picked up by the next scheduled pipeline run. In the worst case, expect up to three hours of delay plus the time it takes the pipeline to reach the relevant step. If you need it sooner, you can manually trigger the relevant job (for example, Products) from the Jobs page — but make sure all upstream dependencies have run recently first.

A customer placed an order on Shopify — how fast does it reach NetSuite, and what retries exist?

The order webhook is real-time: Shopify posts to the integration the instant the order is placed, and the Sales Order creation job is dispatched immediately. The job is configured with **three retry attempts** and a **600-second timeout** per attempt, so transient NetSuite issues are retried automatically. End-to-end, most orders are created in NetSuite within a few seconds to a few minutes.

Can I trigger a single job manually?

Yes. On the **Jobs** page, click the play icon next to any job to run it on demand. This is the right tool for recovering from a mid-pipeline failure or for pushing an urgent change through between scheduled cycles. Follow the dependency order shown on the Jobs page footer — running a downstream job before its prerequisites can leave records incomplete.

The pipeline stopped halfway — will it catch up?

Yes. When a job fails, the remaining jobs for that cycle are skipped, but the scheduler runs the **full pipeline again from Step 1 at the next scheduled tick (three hours later)**. Most transient issues clear on the next run with no operator action needed. You do not need to manually restart a failed pipeline unless you want to recover before waiting for the next scheduled cycle.

I see "Partially Successful" status on a job — what does that mean?

It means some records in that run synced successfully and others did not. For example, out of 500 companies, 498 may have written correctly while 2 encountered errors. Open the job on the Jobs page to see the counts and review the failed records on the Errors page. If the same records continue to fail across multiple runs, escalate to Makro Agency.

Can I change who receives notifications?

Yes. Go to the **Settings** page in the portal. Notification recipients are stored in the application's settings table and can be edited directly there.
